

MIRC701 實驗室研究主題與計畫訪談紀錄表

訪談日期：115 年 6 月 日 時 分

論文名稱：Detecting and Preventing Input Injection in LLM-based Agents with 3-Layer Filtering

(基於因果圖的多源日誌關聯結合大型語言模型驅動之攻擊生命週期補全)

➔ 論文名稱有錯，應該是 HiPert: A Three-Layer Syntax-Semantics-Aware Defense Against Indirect Prompt Injection in LLM-based Code Contexts

受訪者姓名：鄭博涵 (Jack)

一、研究動機 (Motivation):

Q1: Why do you believe traditional Prompt Injection defenses are no longer sufficient for protecting AI Agents? (為什麼你認為傳統 Prompt Injection Defense 已經不足以保護 AI Agent?)

傳統防禦是為「純文字 prompt、已知攻擊分布」設計的，但在程式碼場景中，payload 藏在語法合法的非執行區域 (comment / string / identifier)，讓困惑度偵測和分類器都失效。

三個具體失效原因：

1. 攻擊面從「文字指令」轉移到「程式碼結構」 Code LLM 的輸入不只有 user prompt，還有從 GitHub、Stack Overflow、文件、RAG 系統撈回來的外部程式碼，攻擊者只需污染這些外部來源，不需要接觸開發者或直接操控 prompt。而且污染藏在 comment / string / identifier 等非執行區域——不影響程式執行邏輯，也不觸發靜態分析工具。
2. PPL 困惑度失效：XOXO 和 ITGen 只是把變數名 USE_RAW_QUERIES 改成 RAW_QUERIES，困惑度低、語法合法，看起來和正常的程式碼重構沒有區別。傳統 PPL-based 偵測完全漏判。
3. 學習型分類器失效：CodeGarrison 這類學習型防禦依賴已知攻擊分布，面對新型攻擊 (如 CoTDeceptor、ITGen) 大幅退化，因為它學的是「已知攻擊長什麼樣」，而非「payload 對模型行為造成什麼影響」。

為什麼 Code LLM 特別脆弱：程式碼中的非執行區域 (comment / string / identifier) 是「LLM 看得到、編譯器看不到」的盲點，攻擊者可以把 payload 藏在這裡，繞過傳統的靜態解析工具以及編譯器，但卻影響 LLM 的輸出。

延伸到 Coding Agent (後期場景)：Coding Agent 讓問題更嚴重，因為 Agent 會執行任務——把生成的惡意程式碼真正執行出去，造成 SSH key 洩漏、機密外流等實際危害。而即使沒有 Agent，LLM 產生的含漏洞程式碼本身就已經是嚴重威脅。例如 os.system(rm -rf) 就可以成功透過 ShadowCode 攻擊餵入 LLM，並製造出來，最終刪掉整個目錄檔案

Q2: What incidents or attack cases motivated you to study Input Injection attacks? (什麼事件或案例讓你開始研究 Input Injection?)

我的是 Indirect Prompt Injection，是包含在 Input Injection 裡面，順帶一提 Input

Injection 名詞是由 Bang 同學命名的，業界目前還沒有 Input Injection 這個專有名詞。

而為什麼會研究 Indirect Prompt Injection 是因為：Code LLM 在軟體開發中快速普及，但「開發者無惡意、卻因為引用了外部程式碼中招」這個攻擊模式，在既有防禦框架裡完全是空白，而這些攻擊有個共同點：「攻擊者不需要接觸受害者，只需污染一個會被引用的公開來源」。這讓防禦必須在程式碼進入 LLM context 之前進行偵測及清潔，也就是我做的防禦及偵測框架。

二、問題定義 (Problem Definition):

Q3: What is the major difference between Input Injection and Prompt Injection?

(Input Injection 與 Prompt Injection 最大差異是什麼?)

Indirect Prompt Injection 是 Input Injection 的子集合

Input Injection: 攻擊者污染任何進入 AI Agent 的 input channel，使 agent 將資料誤解為指令，或使其決策、tool use、output 偏離原本意圖。強調 agent 的某個 input channel 被污染

Indirect Prompt Injection: 攻擊者污染外部資料來源，例如網頁、文件、email、RAG context，外部程式碼檔，使 LLM 或 AI Agent 在讀取這些資料時被間接操控，強調使用者本身無惡意，且外部資料中的內容被 LLM 當成 prompt / instruction

Q4: Why are correlated attacks more dangerous than single-channel attacks?

(Correlated Attack 為何比單一通道攻擊更危險?)

不太確定你說的 Correlated Attack 定義是什麼，我論文中應該沒有提到 Correlated Attack

三、方法設計 (Methodology):

Q5: Why did you choose a three-layer architecture instead of a single model?

(為什麼選擇三層式架構，而不是單一模型?)

不同攻擊類型依賴不同訊號，單一模型在「成本」與「涵蓋率」上必然顧此失彼；分層 + early-exit 在 F1 與延遲上同時優於任何單層。讓快速過濾的層先解決多數容易案例，昂貴的層只處理剩餘的困難案例 (early-exit)，若全部只用第三層進行偵測，時間大約是 168ms，若一到三層一起，則會變成 100ms，且 F1 score 從第三層的 0.74 上升到 0.79

Q6: What are the responsibilities of Layer 1, Layer 2, and Layer 3

respectively? (Layer 1、Layer 2、Layer 3 各自負責什麼工作?)

Layer 1 — Syntax-Guided Pre-Filtering (≈7.6ms) 目標：明顯 / 結構型 payload + Flashboom 的 dead decoy

Layer 2 — CST-Guided Dynamic Min-K% Scoring (≈75ms) 目標：ShadowCode / INSEC 的亂碼對抗型 payload

Layer 3 — CST-Guided Node Perturbation Analysis (≈168ms) 目標：XOXO / ITGen 的低困惑度語意型 trigger

Q7: How do the three layers jointly reduce false positives and latency? (三層之間如何降低誤判率與延遲?)

- Layer 1 平滑比率避免短合法識別符因一個異常字元被誤判; regex 保守設計 (寧可放過讓後層處理)。
- Layer 3 surprise-gate 確保只對真正高影響力節點做 dual inference, 不對正常節點亂動

四、創新性 (Novelty):

Q8: What are the key differences between your approach and Sentinel, RAGuard, and ToolParsing? (你的方法與 Sentinel、RAGuard、ToolParsing 最大差異在哪裡?)

面向	AgentSentinel	RAGuard	ToolParsing	HiPert
防禦時機	工具執行後偵測行為異常	檢索階段過濾毒文件	解析 tool output 格式	LLM 讀取 context 之前清潔
偵測訊號	工具行為/輸出的異常	文件的語意相似性/分布	欄位格式是否合法	程式碼語法結構 + 對模型行為的因果影響力
能否抓語意型攻擊	否 (行為層面已為時已晚)	否 (語意型毒文件外觀正常)	否 (payload 藏在合法欄位裡)	是 (Layer 3 node perturbation 直接量影響力)
黑箱相容	需要知道工具執行情況	需要訓練 retriever	需要 agent 架構配合	是, 用本地代理模型, 不需存取 victim 模型

他們三者都在保護「pipeline 的某個特定環節」, 而且都是用「外觀特徵」或「格式合法性」做判斷——對外觀正常、語意隱蔽的攻擊 (XOXO / ITGen) 全部失效。HiPert 作用在「任何 input 進入 LLM 之前」, 並且用「影響力」而非「外觀」來定義惡意, 補上了他們留下的漏洞

Q9: Why can Interaction-Level Defense detect attacks that other methods miss? (為什麼 Interaction-Level Defense 能偵測到其他方法無法發現的攻擊?)

你說的 Interaction-Level Defense 是指 Node Perturbation Analysis?

攻擊者可以偽裝外觀, 但**無法偽裝影響力**。任何能讓模型產生惡意輸出的 trigger, 必然對 model logits 造成可量測的影響——移除它, logits 就會劇烈變化

為什麼其他方法容易漏掉攻擊:

- PPL / naturalness (KillBadCode): identifier 困惑度低、語法合法 → 判「正常」。
- 分類器 (CodeGarrison): 訓練資料裡沒見過這種模式 → 判「正常」。
- 格式檢查 (ToolParsing): identifier 是合法字串 → 判「正常」。
- RAG 過濾 (RAGuard): 整份程式碼的語意分布正常 → 判「正常」

五、實驗與成果 (Evaluation):

Q10: How do you evaluate the effectiveness of your defense system? (你如何評估防禦效果?)

F1 Score, FPR, AUROC, Latency, Clean-Code Utility

Q11: Among ASR, F1 Score, and Latency, which metric is the most important and why? (ASR、F1 Score、Latency 哪一項最重要? 為什麼?)

ASR (目的) > F1 (機制) > Latency (約束)

- **ASR**: 防禦的終極任務是「victim 模型不被攻擊成功」。F1 高但清潔沒中性化真正的 trigger, ASR 還是降不下來——在學術上好看、實務上無用。ASR 直接對應開發者有沒有被真正保護到。
- **F1**: 開發與調參階段最有用的診斷指標, 是達成低 ASR 的手段, 而非目的本身。
- **Latency**: HiPert 是推論前的前處理, 必須維持互動可接受 ($\approx 100\text{ms}$)。這是「門檻條件」, 達標即可, 低延遲本身不等於好防禦。

六、未來工作 (Future Work):

Q12: If attackers know your three-layer framework, how might they bypass it? (如果攻擊者知道你的三層架構, 他可能如何繞過系統?)

三種已測試的 **adaptive attack** 策略:

攻擊策略	目標層	原理	F1 影響
Decoy Injection	Layer 3	注入大量高 surprise 但無害節點, 佔滿候選名額, 把真正 trigger 擠出評估範圍	0.82 $\rightarrow$ 0.74
Copy Trigger	Layer 3	複製相同 trigger 到多個位置, 移除單一節點時 logits 影響被其他副本稀釋	0.82 $\rightarrow$ 0.62 (最大)
Contextual Attack	Layer 3	用基因演算法優化 trigger 周圍上下文, 壓低 surprise 分數讓 gate 跳過它	0.82 $\rightarrow$ 0.66

Q13: Would you consider extending this work to secure Multi-Agent systems in the future? (未來是否考慮支援 Multi-Agent 系統的安全防護?)

會, 這是把 HiPert 往完整 Indirect Prompt Injection 問題延伸的自然方向。先在 Coding Agent 上成功部署及測試, 將我的防禦框架打包成 MCP Tool 並在 Cursor / Claude 裡面使用, 在使用者上傳檔案之前進行檔案偵測及清潔, 清潔後再交給 Cursor / Claude。

在 Multi-Agent 場景, 一個 Agent 的 output 會變成另一個 Agent 的 input (memory、message、tool result), 污染 agent 傳播, 或許可以做這方面清潔

七、最關鍵問題

Q14: If you could describe the value of your thesis in one sentence, what would it be? (如果只能用一句話說明你的論文價值, 你會怎麼說?)

我提出一套推論時運作、無需重訓練且能作用在黑箱商業模型上的三層防禦框架, 在程式碼場景的 Indirect Prompt Injection 問題上, 用語法結構定位加上節點對模型行為的

因果影響力量測，平均 F1 Score 達到 0.80，在不破壞程式功能的前提下，把多種新型 Indirect Prompt Injection 的攻擊成功率從 64 - 75% 降至 10.5 - 21.5%

Q15: What is the fundamental problem you are solving, and why is it worth solving? (你解決的根本問題是什麼？為什麼這個問題值得被解決？)

LLM 與 AI Agent **無法區分「不可信的 input 資料」和「需要執行的指令」**。在程式碼 context 這個 Input Injection 通道中，攻擊者可以把惡意語意藏在語法合法的 comment / string / identifier 裡——編譯器看不到、靜態分析看不到、PPL 抓不到——但 LLM 會把它當成強力信號，產生含惡意邏輯的程式碼或採取危險行動

為什麼值得被解決：

1. **既有防禦有結構性缺口**：PPL 困惑度漏掉語意型攻擊；學習型分類器面對未見攻擊退化；需要改模型的防禦 (SecAlign / StruQ) 無法保護黑箱商用 Agent；AgentSentinel / RAGuard / ToolParsing 各自只保護 pipeline 的一個環節，都不處理程式碼結構層的語意型注入。
2. **影響面廣且即時**：Copilot、Claude Code、Cursor 等 Coding Agent 已是主流開發工具，且主動從公開來源 (GitHub、Stack Overflow) 撈外部程式碼。一個被污染的公開檔案可在開發者毫無互動的情況下波及所有引用它的下游系統。
3. **後果嚴重且可擴展**：受誤導的 Code LLM 產生的惡意程式碼若被執行，攻擊者可取得 SSH key、公司機密乃至開發環境控制權。在 Multi-Agent 場景下，污染可沿 agent 圖傳播，危害範圍進一步放大。